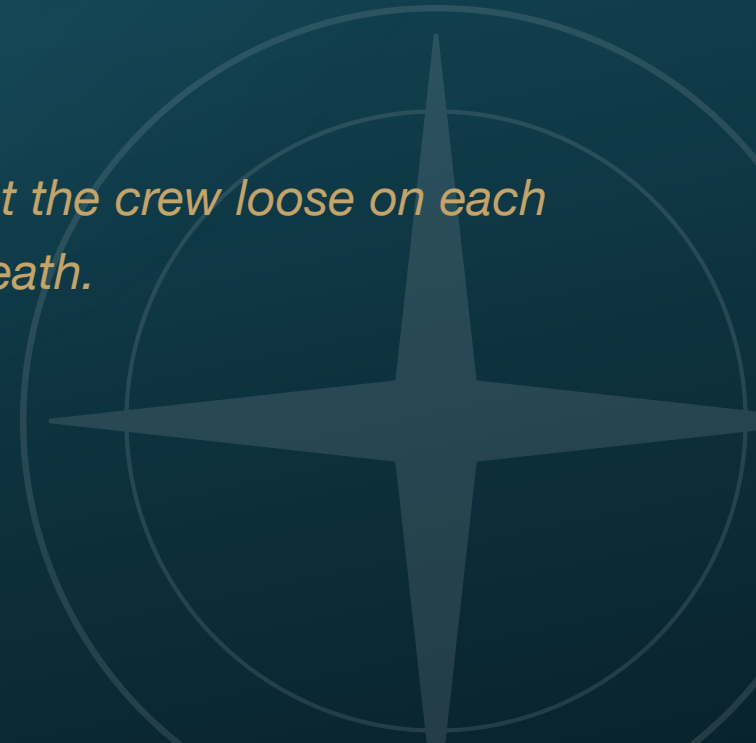


DAY 3 • COLLABORATION PLAYBOOK • SHORT TALK-THROUGH

All Hands on Deck

Turning one cluster into a crew

You ran the sabotage in the lab. Here's the rest of the fleet of ways to set the crew loose on each other — and the platform-engineering twist underneath.



The rule that makes it safe

kubectl is a suggestion.	Git is the law.
live edits heal away	a merged commit sticks

- ♦ Give students **broad rights** in the cluster *and* keep every app **GitOps-managed with self-heal on**.
- ♦ Now collaboration can't break anything permanently: a live edit by a neighbor is **graffiti that heals**, the repo is the only thing that lasts.
- ♦ That single guarantee is what lets you say "*go ahead — delete each other's stuff.*"

Every scenario in this deck rides on that one rule.

Already in the lab: Sabotage & Self-Heal

- ♦ The official one — they just did it.
- ♦ *"With the rights the Admiral granted you, go delete a **crewmate's** deployment."*
- ♦ Their neighbor's ArgoCD hauls it back within seconds. You **cannot win against someone else's Captain's Log.**
- ♦ The takeaway, out loud: *"So how do you actually change a crewmate's fleet? You don't touch their cluster — you change their law."* → which is the next slide.

THE MENU

Set the crew loose

\o/ \o/ \o/ \o/
| | | |
/ \ / \ / \ / \
~~~~~  
one cluster · many hands

*Four more ways to make thirty people one crew.*

## 1 • The PR Raid (*the constructive twin*)

---

**Move:** open a pull request against a *neighbor's* Gitea repo.

- ♦ Add a `readinessProbe`. Bump replicas. Fix a missing label.
- ♦ The owner **reviews and merges** → ArgoCD ships it → it **sticks**.
- ♦ Vandalism heals away; a **merged PR lasts**. Both poles of "Git is the law," felt back to back.

*You can't change my cluster. You can propose a change to my law.*

## 2 • The Chain Reaction (*one organism*)

---

**Move:** wire each student's app to call the **next crew's** Service.

- ♦ A ring: your frontend → your right-hand neighbor's backend.
- ♦ Someone deletes their deployment — the break **ripples downstream** — then self-heal restores the whole chain.
- ♦ *Your uptime is my uptime.* Teaches cross-namespace Services & DNS.

*Bonus: it's the perfect Day 4 on-ramp — the Pirate breaks one node and the blast propagates.*

### 3 • The Town Square (*the shared artifact*)

---

**Move:** one shared namespace with a guestbook / leaderboard.

- ♦ Every student's chart deploys **one tile** into it via their own ArgoCD app.
- ♦ The picture is only complete when **all thirty** are synced.
- ♦ Delete the town-square app and *everyone's* contribution vanishes — then GitOps brings it all back.

*The most visual whole-room moment. Take the photo.*

## 4 • Fix My Broken Chart (*collaborative debugging*)

---

**Move:** everyone plants a bug, then swaps repos.

- ♦ Break your own chart — bad indent, wrong `.Values` ref, image typo.
- ♦ Push it; ArgoCD flips to **Degraded**.
- ♦ Hand the repo to a neighbor to **diagnose and fix via PR**.
- ♦ Teaches `helm template`, reading ArgoCD's error states, and `helm lint`.

*Pairs naturally with the PR Raid — same workflow, opposite intent.*

## THE IDEA UNDERNEATH

# Division of Labor

produce —▶ consume —▶ verify  
(build it) (use it) (guard it)

*A platform team doesn't make everyone build everything.*

# Platform engineering is a relay, not a solo

---

- ♦ On a real platform team, the work **splits by role**:
  - someone **produces** a reusable thing (a paved road),
  - others **consume** it (self-service, no internals),
  - others **verify & guard** it (tests, policy, scans).
- ♦ Assign those three roles across a crew and **your classroom becomes a platform team** — each student feels a different seat.
- ♦ The next three slides are the same pattern, three different "things."

## A → B → C • The Reusable Workflow

| Seat    | Student | Does                                                                                                       |
|---------|---------|------------------------------------------------------------------------------------------------------------|
| Produce | A       | Authors a CI <b>workflow</b> (Gitea Actions / Argo <code>WorkflowTemplate</code> ) — build, tag, push.     |
| Consume | B       | <b>Uses</b> it from their own repo: <code>uses: A/ci-workflows/build@v1</code> . Never rewrites it.        |
| Verify  | C       | Writes the <b>tests</b> that keep it honest — a sample repo + assertions that fail if A's workflow breaks. |

*One pipeline, authored once, consumed by the whole crew, guarded by a third hand. That's a platform.*

## A → B → C • The Golden Helm Chart

| Seat    | Student | Does                                                                                                             |
|---------|---------|------------------------------------------------------------------------------------------------------------------|
| Produce | A       | Publishes a parameterized chart to Harbor: <code>helm push stack.tgz oci://harbor.../charts</code> .             |
| Consume | B       | Installs it with <b>their own</b> <code>values.yaml</code> — turns the knobs, writes <b>zero</b> templates.      |
| Verify  | C       | Adds a <code>values.schema.json</code> + <code>helm test</code> that <b>rejects bad inputs</b> before they ship. |

- ♦ This is today's whole lesson, socialized: the chart *is* the golden path.

*B self-services on A's blueprint. C makes the blueprint safe to hand out.*

## A → B → C • The Golden Base Image

| Seat    | Student | Does                                                                                                                    |
|---------|---------|-------------------------------------------------------------------------------------------------------------------------|
| Produce | A       | Builds a hardened base image, pushes to Harbor: <code>harbor.../base:1.0</code> .                                       |
| Consume | B       | Builds their app <code>FROM harbor.../base:1.0</code> — inherits the blessed toolchain.                                 |
| Verify  | C       | Runs <b>Trivy</b> scans + writes a <b>Kyverno</b> policy that <i>blocks</i> any image not built from the approved base. |

*Supply chain as a team sport: a golden image, everyone downstream of it, a gate that enforces it.*

## Where this goes (name-drop, don't teach)

---

Same **produce** → **consume** → **verify** relay, one level up:

- ♦ **Scaffolding catalog** (Backstage software templates) — A writes the "blessed new-service" template; B scaffolds from it in one command; C gates it in CI.
- ♦ **Self-service infrastructure** (Crossplane) — A defines a Database API; B writes a 5-line claim and *gets* a Postgres; C sets the quota and policy on what can be claimed.

*The seats never change. Only the size of the thing on the conveyor belt.*

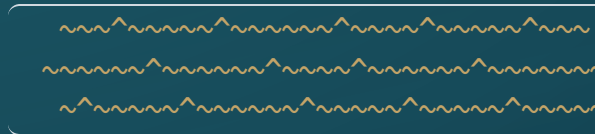
# Instructor Superpower

---

## ASSIGN THE SEATS — AND THE ROOM BECOMES A PLATFORM TEAM

- ♦ Pick a scenario, hand out the roles, and the collaboration *is* the curriculum.
- ♦ Your students stop being thirty solo operators and start being **one crew on one platform** — produce, consume, verify.
- ♦ That is the experience you came here to feel today. It's also the one you can hand your own students next term.

# All hands.



*One cluster. Many crews. Git keeps the peace.*

